iCEV - INSTITUTO DE ENSINO SUPERIOR
CURSO DE ENGENHARIA DE SOFTWARE — 1º PERÍODO (2026.2)
CÁSSIO FILIPE MEIRELES DE CARVALHO BRAGA (MATRÍCULA: 0031236)
LUKAS SOUZA MELO (MATRÍCULA: 0031163)

IMPLEMENTAÇÃO DE ESTRUTURAS LINEARES, NÃO-LINEARES E ALGORITMOS DE ORDENAÇÃO APLICADOS A PROBLEMAS REAIS DO COTIDIANO

Trabalho prático avaliativo (P2) apresentado à disciplina de Algoritmos e Estruturas de Dados I, ministrada pelo Prof. Hilson Silva, no Instituto de Ensino Superior iCEV.

Turma: Snyder

FICHA OBRIGATÓRIA DE IDENTIFICAÇÃO

Campo Obrigatório	Informação da Dupla
Instituição	Instituto de Ensino Superior iCEV
Disciplina / Avaliação	Algoritmos e Estruturas de Dados I — Avaliação P2
Professor	Prof. Hilson Silva
Nome da Turma / Período	SNYDER — 1º Período (2026-2)
Linguagens Utilizadas	Java (JDK 17+) e C (Padrão C11)
Aluno 1	Cássio Filipe Meireles De Carvalho Braga
Aluno 2	Lukas Souza Melo
Matrícula Aluno 1	0031236
Matrícula Aluno 2	0031163

TERESINA - PI

2026

SUMÁRIO

FICHA OBRIGATÓRIA DE IDENTIFICAÇÃO **1**

1. INTRODUÇÃO E ARQUITETURA DO PROJETO **3**

2. ESTRUTURAS DE DADOS LINEARES **5**

2.1 FILAS (QUEUES) **5**

2.2 PILHAS (STACKS) **7**

2.3 LISTAS ENCADEADAS (LINKED LISTS) **9**

3. ESTRUTURAS DE DADOS NÃO-LINEARES **11**

3.1 ÁRVORE BINÁRIA DE BUSCA (BST) **11**

3.2 ÁRVORE BALANCEADA (AVL) **13**

3.3 GRAFOS (GRAPHS) **14**

3.4 TABELA HASH (HASH TABLE) **16**

4. ALGORITMOS DE ORDENAÇÃO **18**

4.1 BUBBLE SORT **18**

4.2 SELECTION SORT **20**

4.3 INSERTION SORT **21**

4.4 MERGE SORT **23**

4.5 QUICK SORT **24**

5. SÍNTESE DE COMPLEXIDADE, DIFICULDADES E CONCLUSÃO GERAL **26**

5.1 Síntese Comparativa de Complexidade e Validação **26**

5.2 Dificuldades Encontradas e Soluções Adotadas **28**

5.3 Conclusão sobre a Aplicabilidade das Estruturas em Problemas Reais **29**

6. REFERÊNCIAS BIBLIOGRÁFICAS **29**

1. INTRODUÇÃO E ARQUITETURA DO PROJETO

O estudo de Algoritmos e Estruturas de Dados é fundamental para o desenvolvimento de soluções computacionais eficientes, pois permite compreender como as informações devem ser modeladas, armazenadas, indexadas e processadas na memória principal para solucionar problemas reais sob diferentes restrições de tempo e espaço.

Este relatório técnico documenta a implementação integral das 12 categorias de estruturas de dados e algoritmos de ordenação aplicadas às 24 situações-problema reais do cotidiano propostas no edital da disciplina de Algoritmos e Estruturas de Dados I (2026.2). Todas as estruturas e rotinas de ordenação foram construídas do zero, sem recorrer a coleções prontas ou métodos nativos de ordenação das bibliotecas padrão, com foco na compreensão rigorosa de seu funcionamento interno.

O projeto contempla três grandes eixos: Estruturas de Dados Lineares (Filas, Pilhas e Listas Encadeadas), Estruturas de Dados Não-Lineares (Árvore Binária de Busca — BST, Árvore Balanceada — AVL, Grafos e Tabela Hash) e Algoritmos de Ordenação (Bubble Sort, Selection Sort, Insertion Sort, Merge Sort e Quick Sort). Para cada uma das 12 categorias foram desenvolvidas duas aplicações completas acompanhadas de justificativa técnica de escolha, detalhamento de implementação, demonstração com dados realistas de entrada e saída, análise assintótica de complexidade de tempo e espaço e validação por testes.

O desenvolvimento foi dividido de forma equilibrada entre os dois integrantes da dupla, abrangendo seis categorias (12 situações-problema) em Java e seis categorias (12 situações-problema) em C:

Cássio Filipe Meireles De Carvalho Braga (Java — diretório java(Cássio)/): Implementação de Filas (Situações 1 e 2), Árvore Binária de Busca — BST (Situações 7 e 8), Árvore Balanceada — AVL (Situações 9 e 10), Bubble Sort (Situações 15 e 16), Insertion Sort (Situações 19 e 20) e Merge Sort (Situações 21 e 22), além de 7 suítes de testes automatizados (verificar.sh) totalizando 107 casos de teste aprovados, 0 pendentes e 0 falhas.

Lukas Souza Melo (C — diretório c(Lukas)/ — subpastas Pilha/, Lista_Encadeada/, Grafo/, Tabela_Hash/, Selection_Sort/, Quick_Sort/): Implementação de Pilhas (Situações 3 e 4), Listas Encadeadas (Situações 5 e 6), Grafos com Dijkstra e BFS (Situações 11 e 12), Tabela Hash com encadeamento externo e hash DJB2 (Situações 13 e 14), Selection Sort (Situações 17 e 18) e Quick Sort (com pivô por Mediana-de-Três na Situação 23 e particionamento de Lomuto na Situação 24), contando com menus interativos e simulações automatizadas completas.

Mapa Consolidado das 12 Categorias e 24 Situações-Problema

#	Categoria	Situação-Problema do Edital	Ling.	Arquivo / Classe Principal
1	2.1 Fila	Sistema de Atendimento de uma Clínica Médica (15 pacientes)	Java	fila.Clinica
2	2.1 Fila	Fila de Impressão em Laboratório (12 trabalhos + cancelar último)	Java	fila.FilaImpressao
3	2.2 Pilha	Sistema de Desfazer/Refazer (Undo/Redo) em Editor de Texto	C	c(Lukas)/Pilha/Sistema de Undo.c
4	2.2 Pilha	Verificação de Expressões (), [] e {} com posição do erro	C	c(Lukas)/Pilha/Verificação de Sistemas.c
5	2.3 Lista Enc.	Gerenciamento de Playlist de Músicas (12 músicas)	C	c(Lukas)/Lista_Encadeada/playlist.c
6	2.3 Lista Enc.	Controle de Estoque de Loja (simulação semanal de 7 dias)	C	c(Lukas)/Lista_Encadeada/estoque.c
7	3.1 BST	Cadastro e Busca de Alunos por Matrícula (20 alunos)	Java	bst.CadastroAlunos
8	3.1 BST	Sistema de Autocomplete de Nomes de Produtos (30 produtos)	Java	bst.AutocompleteProdutos
9	3.2 AVL	Índice de CPF para Sistema de RH (25 funcionários)	Java	avl.IndiceCpf
10	3.2 AVL	Ranking de Jogadores em Jogo Online (30 jogadores + Top-K)	Java	avl.RankingJogadores
11	3.3 Grafo	Roteamento de Entregas (Lista de Adjacência + Dijkstra, 10 bairros + CD Central = 11 vértices)	C	c(Lukas)/Grafo/Entregas.c
12	3.3 Grafo	Rede de Amizades em Rede Social (15 usuários + BFS graus)	C	c(Lukas)/Grafo/rede_social.c
13	3.4 Hash	Cadastro de Usuários com Login Rápido O(1) (20 usuários + DJB2)	C	c(Lukas)/Tabela_Hash/login.c
14	3.4 Hash	Contagem de Frequência de Palavras em Texto Técnico de Demonstração	C	c(Lukas)/Tabela_Hash/frequencia.c
15	4.1 Bubble	Ordenação de Notas de uma Turma Pequena (20 alunos vs Insertion)	Java	bubblesort.NotasTurma
16	4.1 Bubble	Ordenação de Produtos por Preço em Vitrine Pequena (12 itens)	Java	bubblesort.ProdutosPreco
17	4.2 Selection	Seleção dos Melhores Candidatos em Processo Seletivo (Top-N)	C	c(Lukas)/Selection_Sort/processo_seletivo.c
18	4.2 Selection	Organização de Tarefas por Prioridade 1 a 5 (14 tarefas)	C	c(Lukas)/Selection_Sort/tarefas.c
19	4.3 Insertion	Inserção de Novas Cartas em Jogo de Cartas Ordenado (13 cartas)	Java	insertionsort.JogoCartas
20	4.3 Insertion	Atualização de Ranking de Vendedores do Mês (10 vendedores, 7d)	Java	insertionsort.RankingVendedores
21	4.4 Merge	Ordenação de Histórico de Transações	Java	mergesort.TransacoesBancarias

2. ESTRUTURAS DE DADOS LINEARES

As estruturas de dados lineares organizam os elementos de forma sequencial, na qual cada registro possui, no máximo, um predecessor e um sucessor imediato. São essenciais para o controle de fluxos de atendimento, reversão de histórico de operações e armazenamento dinâmico de coleções.

2.1 FILAS (QUEUES)

2.1.1 Conceito e justificativa da escolha

A fila é uma estrutura de dados linear regida pela política FIFO (First-In, First-Out), segundo a qual o primeiro elemento inserido é obrigatoriamente o primeiro a ser removido. Essa propriedade torna a estrutura ideal para cenários nos quais a ordem cronológica de chegada deve ser estritamente respeitada.

Neste trabalho, a fila é aplicada em duas situações: o gerenciamento do atendimento ambulatorial de pacientes em uma clínica médica e o escalonamento de trabalhos enviados para um servidor de impressão em laboratório de informática. Em ambos os casos, a escolha da fila garante justiça, previsibilidade de atendimento e operações de enfileiramento e desenfileiramento em tempo constante O(1).

2.1.2 Implementação da estrutura

A estrutura foi implementada na classe genérica Fila<T> (java(Cássio)/fila/Fila.java) utilizando um vetor circular de capacidade fixa para 20 elementos (Object[20]) controlado pelos atributos inicio, fim e quantidade. O avanço dos ponteiros utiliza aritmética modular (indice + 1) % elementos.length, reaproveitando posições liberadas após remoções sem necessidade de deslocar elementos no vetor.

enqueue(T elemento): Insere um elemento válido no final da fila em O(1) e avança o índice fim circularmente.

dequeue(): Remove e retorna o elemento posicionado em inicio em O(1), avançando o ponteiro inicial.

front(): Consulta o próximo elemento a ser atendido sem removê-lo da estrutura em O(1).

removerUltimo(): Cancela e remove o último elemento enviado que ainda aguarda no final da fila recuando fim com (fim - 1 + elementos.length) % elementos.length em O(1).

isEmpty() e size(): Informam se a fila está vazia e a quantidade exata de elementos aguardando em O(1).

listar(): Retorna a sequência atual de elementos em ordem de chegada sem alterar o estado da fila em O(n).

2.1.3 Situação Problema 1 - Sistema de atendimento de uma clínica médica

Descrição e resultado da execução

A aplicação fila.Clinica simula um dia de atendimento ambulatorial com 15 pacientes chegando em intervalos de 7 minutos a partir das 08:05 (Ana Lima às 08:05, Bruno Costa às 08:12, Carla Alves às 08:19 até Olivia Gomes às 09:43). Cada consulta possui duração estimada de 10 minutos. O sistema permite registrar novas chegadas, chamar o próximo paciente, consultar o início e o tamanho da fila, e emitir o relatório final com a ordem de atendimento e o tempo médio de espera.

Indicador da Simulação (Clínica Médica)	Resultado Observado
Quantidade de pacientes atendidos	15 pacientes
Duração estimada de cada atendimento	10 minutos
Primeiro paciente atendido (horário / espera)	Ana Lima (chegada 08:05 / chamado 08:05 / espera 0 min)
Segundo paciente atendido (horário / espera)	Bruno Costa (chegada 08:12 / chamado 08:15 / espera 3 min)
Último paciente atendido (horário / espera)	Olivia Gomes (chegada 09:43 / chamado 10:25 / espera 42 min)
Tempo médio estimado de espera	21,00 minutos

Análise do resultado

Os resultados comprovam que a fila preservou rigorosamente a ordem de chegada (FIFO). Como os pacientes chegam a cada 7 minutos e cada consulta dura 10 minutos, há um acúmulo progressivo de 3 minutos por paciente, elevando a espera de 0 minutos no primeiro atendimento para 42 minutos no décimo quinto, resultando na média aritmética exata de 21,00 minutos.

2.1.4 Situação Problema 2 - Fila de impressão em laboratório de informática

Descrição e resultado da execução

A aplicação fila.FilaImpressao gerencia trabalhos de impressão contendo nome do aluno, nome do arquivo e número de páginas. A simulação carrega 12 trabalhos (totalizando 75 páginas), demonstra a funcionalidade de cancelamento do último trabalho enviado (Lucas - seminario.pdf - 9 páginas) por meio da operação removerUltimo() e processa os trabalhos restantes na ordem de envio.

Indicador da Simulação (Fila de Impressão)	Resultado Observado
Trabalhos enviados inicialmente	12 trabalhos (75 páginas)
Cancelamento do último trabalho enviado (Opção 6)	Lucas - seminario.pdf (9 páginas canceladas)
Trabalhos efetivamente impressos (Opção 5)	11 trabalhos (de Ana - algoritmos.pdf a Karen - tabela.pdf)
Total de páginas impressas após cancelamento	66 páginas (ou 75 páginas sem cancelamento)
Respeito à política FIFO de impressão	Sim (ordem estrita de envio)

Análise do resultado

A execução validou tanto o processamento FIFO padrão do spooler de impressão quanto o requisito de cancelamento do último trabalho enviado enquanto ainda aguarda na fila, ajustando o índice circular fim em tempo constante sem afetar os trabalhos anteriores.

2.1.5 Análise de complexidade

As operações enqueue, dequeue, front, removerUltimo, isEmpty e size possuem complexidade de tempo O(1) no melhor, médio e pior casos, pois realizam apenas cálculos aritméticos modulares e acesso direto ao array. A operação listar percorre os n elementos presentes, apresentando complexidade de tempo O(n). Em relação ao espaço, a estrutura aloca O(N) para a capacidade máxima configurada (N = 20) e utiliza O(1) de memória auxiliar por operação.

2.1.6 Testes realizados e considerações sobre as aplicações

A suíte automatizada fila.TesteFila validou 16 casos de teste (incluindo fila vazia, inserção/remoção unitária e múltipla, giro circular completo do vetor, cancelamento do último elemento com recuo do índice zero, rejeição de nulos e verificação das bases da clínica e da impressora), obtendo 16 aprovações sem falhas.

2.2 PILHAS (STACKS)

2.2.1 Conceito e justificativa da escolha

A pilha é uma estrutura linear baseada no princípio LIFO (Last-In, First-Out), no qual o último elemento inserido é o primeiro a ser removido. Todas as inserções (push) e remoções (pop) ocorrem exclusivamente em uma única extremidade denominada topo.

A escolha da pilha para o sistema de Desfazer/Refazer (Undo/Redo) em um editor de texto é natural porque a ação mais recente executada pelo usuário deve ser obrigatoriamente a primeira a ser revertida. Da mesma forma, na verificação de expressões matemáticas com parênteses, colchetes e chaves, o último delimitador de abertura encontrado precisa ser o primeiro a ser fechado, tornando a pilha a estrutura canônica para análise sintática de escopos aninhados.

2.2.2 Implementação da estrutura

As duas aplicações foram desenvolvidas em linguagem C no diretório c(Lukas)/Pilha/. Em c(Lukas)/Pilha/Sistema de Undo.c, a estrutura Pilha armazena um vetor de registros Acao (contendo tipo e texto) controlado pelo índice inteiro topo (inicializado em -1). Duas instâncias de pilha operam em conjunto: a pilha undo registra as ações realizadas e a pilha redo recebe as ações desfeitas, permitindo refazê-las. Quando uma nova ação original é inserida, a pilha redo é reinicializada para manter a consistência temporal.

Em c(Lukas)/Pilha/Verificação de Sistemas.c, a pilha armazena estruturas ItemPilha contendo o caractere de abertura ('(', '[' ou '{') e o índice exato (posicao) onde ele ocorreu na string. A função validarExpressao percorre a expressão caractere a caractere: ao encontrar um símbolo de abertura, empilha-o com sua posição; ao encontrar um fechamento, verifica se a pilha está vazia ou se o topo não corresponde ao par esperado via ehPar(abertura, fechamento), emitindo um relatório detalhado com a posição exata do primeiro desbalanceamento.

2.2.3 Situação Problema 3 - Sistema de desfazer (Undo) em editor de texto simples

Descrição e resultado da execução

O programa c(Lukas)/Pilha/Sistema de Undo.c oferece menu interativo e uma simulação automatizada (Opção 7) que executa 12 ações sucessivas de edição (digitação de texto, formatação de fonte/estilo/cor e exclusão), realiza 4 operações seguidas de Desfazer (pop da pilha undo para a pilha redo) e, em seguida, executa 2 operações de Refazer (pop da pilha redo de volta para a pilha undo).

Etapa da Simulação (Editor de Texto)	Operação na Pilha	Resultado Observado
Passo 1: Registro de ações	12 operações push na pilha Undo	12 ações empilhadas (topo na posição 12)
Passo 2: 1º Undo	pop(undo) -> push(redo)	Revertida: [Exclusao] 'Fim da primeira secao.'
Passo 2: 2º Undo	pop(undo) -> push(redo)	Revertida: [Digitacao] 'Fim da primeira secao.'
Passo 2: 3º Undo	pop(undo) -> push(redo)	Revertida: [Formatacao] 'Fonte alterada para tamanho 12'
Passo 2: 4º Undo	pop(undo) -> push(redo)	Revertida: [Digitacao] ' usando Pilhas em C.'
Passo 3: 1º e 2º Redo	2x pop(redo) -> push(undo)	Refeitas as ações ' usando Pilhas em C.' e 'Fonte tamanho 12'
Estado final das pilhas	Listagem de controle	10 ações ativas na pilha Undo e 2 ações disponíveis na pilha Redo

Análise do resultado

A simulação demonstrou a reversão passo a passo na ordem inversa exata da execução (LIFO), bem como a recuperação de estados desfeitos com o auxílio da segunda pilha (Redo), atendendo integralmente aos requisitos obrigatórios e opcionais do enunciado.

2.2.4 Situação Problema 4 - Verificação de expressões com parênteses, colchetes e chaves

Descrição e resultado da execução

O programa c(Lukas)/Pilha/Verificação de Sistemas.c avalia expressões matemáticas ou trechos de código contendo (), [] e {}. Na Opção 2 (simulação automática), são executados sete casos de teste — os quatro obrigatórios do enunciado ({ [( )] }, ([ )], ((())), ([)]) e três casos adicionais (([]], {[a + (b * c)] - d} e ((a + b) * [c - d]). Na Opção 1 (entrada interativa manual), foi verificado adicionalmente o caso de delimitador de fechamento com a pilha vazia (a + b) + c), conforme diferenciado na tabela a seguir:

Expressão e Modo de Execução	Status Retornado	Diagnóstico e Posição Exata do Desbalanceamento
{ [( )] } [Opção 2 — Auto]	CORRETA	Expressão perfeitamente balanceada
([ )] [Opção 2 — Auto]	INCORRETA	Caractere ')' na posição 4 não combina com '[' aberto na posição 2
((())) [Opção 2 — Auto]	CORRETA	Expressão perfeitamente balanceada
([)] [Opção 2 — Auto]	INCORRETA	Caractere ')' na posição 2 não combina com '[' aberto na posição 1
([]] [Opção 2 — Auto]	INCORRETA	Caractere ']' na posição 3 não combina com '(' aberto na posição 0
{[a + (b * c)] - d} [Opção 2 — Auto]	CORRETA	Expressão algébrica perfeitamente balanceada
((a + b) * [c - d] [Opção 2 — Auto]	INCORRETA	Delimitador '(' aberto na posição 0 não foi fechado até o final
a + b) + c [Opção 1 — Manual]	INCORRETA	Caractere ')' na posição 6 não possui delimitador de abertura

Análise do resultado

O armazenamento conjunto do caractere e de seu índice na pilha permitiu não apenas classificar a expressão como válida ou inválida, mas também apontar o índice exato do erro sintático tanto em cruzamentos indevidos de delimitadores quanto em aberturas pendentes ao final da string.

2.2.5 Análise de complexidade

As operações elementares da pilha (empilhar, desempilhar, topo, vazia e cheia) possuem complexidade de tempo O(1) e espaço auxiliar O(1). Na validação de expressões de comprimento n, cada caractere é lido uma única vez e empilhado/desempilhado no máximo uma vez, resultando em complexidade de tempo O(n) e complexidade de espaço O(n) no pior caso.

2.2.6 Testes realizados e considerações sobre as aplicações

Os dois programas em C foram compilados com GCC sem avisos e verificados tanto pelas rotinas de simulação integradas aos menus (Opção 7 no editor de texto e Opção 2 no validador de expressões) quanto por execuções interativas adicionais para conferir estados de pilha vazia, limite de capacidade, ciclos sucessivos de Undo/Redo, limpeza da pilha Redo após nova ação e expressões com erros em diferentes posições.

2.3 LISTAS ENCADEADAS (LINKED LISTS)

2.3.1 Conceito e justificativa da escolha

Listas encadeadas armazenam coleções de elementos em nós alocados dinamicamente na memória, interligados por ponteiros. Ao contrário de vetores estáticos, não exigem pré-alocação contígua e permitem inserir, remover ou mover elementos em posições intermediárias ajustando apenas referências.

Na playlist de músicas, a lista encadeada é ideal porque o usuário adiciona faixas no início, no fim ou em posições específicas, remove músicas e reordena a sequência de reprodução com frequência. No controle de estoque de uma loja, novos produtos entram no catálogo e itens esgotados precisam ser removidos da lista ativa dinamicamente ao longo da semana.

2.3.2 Implementação da estrutura

Implementadas em C no diretório c(Lukas)/Lista_Encadeada/. Em playlist.c, cada nó Musica contém titulo, artista, duracao e o ponteiro proximo, gerenciados pelo descritor Playlist (ponteiro inicio e contador tamanho). Foram implementadas as funções inserirInicioDados, inserirFimDados, inserirPosicaoDados, removerPorTitulo, buscarPorTitulo, moverMusica (que desconecta o nó da posição de origem e o reinsere na posição de destino sem realocar memória) e listar.

Em estoque.c, cada nó Produto armazena codigo, nome, quantidade, preco e proximo. O módulo disponibiliza cadastrarDados, atualizarQtd (que remove automaticamente o produto da lista ativa caso a quantidade chegue a zero), removerProduto, buscarPorCodigo, buscarPorNome, listarEstoqueBaixo e listarTudo, liberando a memória com free() em todas as remoções.

2.3.3 Situação Problema 5 - Gerenciamento de playlist de músicas

Descrição e resultado da execução

A simulação automatizada de c(Lukas)/Lista_Encadeada/playlist.c (Opção 8) constrói uma playlist com 12 músicas utilizando 10 inserções no fim (de Bohemian Rhapsody a Tempo Perdido), 1 no início (Aquarela) e 1 em posição intermediária específica (Evidencias na posição 4), executa buscas por título, move a música Tempo Perdido da posição 12 para a posição 2 e remove a faixa Imagine.

Operação Executada na Playlist	Parâmetros de Entrada	Resultado Observado
Criação inicial da playlist	10 no fim, 1 no início, 1 na posição 4	12 músicas ativas (59,98 min totais)
Busca por título existente	'Evidencias'	Encontrada na posição 4 (Chitãozinho & Xororó - 4,65 min)
Busca por título inexistente	'Inexistente'	Música não encontrada na playlist
Reordenação (mover posição)	Posição 12 ('Tempo Perdido') -> Posição 2	Ordem atualizada mantendo 12 músicas
Remoção por título	'Imagine' (John Lennon)	Nó desalocado; playlist final com 11 músicas (56,91 min)

Análise do resultado

A execução confirmou a flexibilidade da lista encadeada para manipular posições arbitrárias: tanto a inserção na 4ª posição quanto a transferência da 12ª para a 2ª posição foram concluídas apenas reencadeando ponteiros.

2.3.4 Situação Problema 6 - Controle de estoque de uma pequena loja

Descrição e resultado da execução

A simulação de c(Lukas)/Lista_Encadeada/estoque.c (Opção 8) reproduz 7 dias (Segunda a Domingo) de operação de uma loja de materiais de construção, iniciando com 6 produtos cadastrados no 1º dia, registrando saídas para obras, cadastro de 2 novos materiais no 3º dia, remoção automática de produto esgotado no 4º dia, buscas por código e nome no 5º dia, vendas e reposição no 6º dia e emissão do relatório de produtos abaixo do estoque mínimo (< 10 unidades) no 7º dia.

Dia / Evento da Semana	Movimentação no Estoque	Resultado na Lista Encadeada
Dia 1 (Segunda)	Cadastro inicial do catálogo	6 produtos ativos na lista (Códigos 101 a 106)
Dia 2 (Terça)	Saída de Cimento (Cod 101) e Argamassa (Cod 102)	Cimento atualizado para 18 un. e Argamassa para 7 un.
Dia 3 (Quarta)	Cadastro de Tubo PVC 100mm (Cod 107) e Disjuntor 40A (Cod 108)	Lista expandida dinamicamente para 8 produtos ativos
Dia 4 (Quinta)	Venda total de Caixa D'Água 1000L (Cod 105)	Quantidade = 0; produto removido automaticamente da lista ativa
Dia 5 (Sexta)	Consultas no sistema por código (103) e por nome (Tubo PVC)	Tinta Acrílica 18L (Cod 103) e Tubo PVC 100mm (Cod 107) localizados
Dia 6 (Sábado)	Vendas de fim de semana (Cod 103, 104, 108) e reposição de Cimento	Tinta cai para 4 un., Tijolo para 3 un., Disjuntor para 6 un.; Cimento sobe para 50 un.
Dia 7 (Domingo)	Auditoria de estoque mínimo (< 10 un.)	4 itens listados em alerta (Cod 108, 104, 103 e 102); 7 produtos ativos

Análise do resultado

O controle de estoque demonstrou o gerenciamento do ciclo de vida completo dos nós em memória dinâmica, incluindo a exclusão automática de itens cuja quantidade atinge zero e a filtragem linear por limite mínimo de reposição.

2.3.5 Análise de complexidade

A inserção no início da lista encadeada possui complexidade de tempo O(1). As operações de inserção no fim, inserção em posição específica, remoção por chave, busca sequencial, reordenação e listagem percorrem até n nós, apresentando complexidade de tempo O(n) no caso médio e pior caso. O consumo de memória é O(n), proporcional ao número exato de elementos ativos, com espaço auxiliar O(1) por operação.

2.3.6 Testes realizados e considerações sobre as aplicações

Ambos os módulos em C foram verificados por meio das simulações automatizadas (Opção 8 em playlist.c e Opção 8 em estoque.c, que reproduzem os fluxos das tabelas acima) e por testes manuais no menu interativo contemplando listas vazias, remoção do primeiro e do último nó, busca de chaves inexistentes, tentativa de inserção em posições inválidas e desalocação ao encerrar o programa.

3. ESTRUTURAS DE DADOS NÃO-LINEARES

As estruturas de dados não-lineares organizam os elementos de forma hierárquica (árvores), em rede de relacionamentos (grafos) ou por dispersão de chaves (tabelas hash), viabilizando buscas logarítmicas ou constantes e a modelagem de topologias complexas.

3.1 ÁRVORE BINÁRIA DE BUSCA (BST)

3.1.1 Conceito e justificativa da escolha

A Árvore Binária de Busca (BST) organiza chaves hierarquicamente de modo que todos os nós da subárvore esquerda possuam chaves estritamente menores que a do nó pai, e todos os nós da subárvore direita possuam chaves maiores. Essa propriedade permite localizar, inserir e remover registros em tempo médio O(log n), além de produzir listagens ordenadas via percurso em-ordem sem necessidade de reordenar toda a coleção.

No cadastro acadêmico, a BST mantém os alunos indexados pela matrícula. No sistema de autocomplete de produtos, a ordenação lexicográfica da BST permite podar subárvores inteiras durante a consulta por prefixo.

3.1.2 Implementação da estrutura

Implementada na classe genérica BST<K extends Comparable<K>, V> (java(Cássio)/bst/BST.java). Disponibiliza inserir (atualizando o valor caso a chave já exista), buscar, remover (tratando nós folha, nós com um filho e nós com dois filhos por substituição pelo sucessor em-ordem menorNo(atual.direita)), listarEmOrdem e listarPorPrefixo.

No método listarPorPrefixo, realiza-se um percurso limitado com poda: se a chave do nó atual for alfabeticamente menor que o prefixo, toda a subárvore esquerda é descartada; se a chave ultrapassar o prefixo (não começando por ele e sendo maior), toda a subárvore direita é descartada.

3.1.3 Situação Problema 7 - Cadastro e busca de alunos por matrícula

Descrição e resultado da execução

A aplicação bst.CadastroAlunos cadastra 20 estudantes (matrículas 1001 a 1020 inseridas fora de ordem sequencial para formar uma árvore ramificada), realiza buscas por matrículas existentes e inexistentes, remove um registro e exibe a listagem ordenada por percurso em-ordem.

Operação na BST (Cadastro de Alunos)	Entrada Utilizada	Resultado Observado
Carga inicial da base	20 alunos (matrículas 1001 a 1020)	20 nós inseridos na árvore BST
Percurso em-ordem (listarEmOrdem)	Árvore completa	Listagem crescente de 1001 (Ana Lima) a 1020 (Samuel Pinto)
Busca por matrícula existente	Matrícula 1001	Encontrado: 1001 - Ana Lima - Computação
Busca por matrícula inexistente	Matrícula 999999	Matrícula não encontrada
Remoção de registro	Matrícula 1008 (Bruno Costa)	Removido mantendo a propriedade de ordem dos 19 alunos restantes

Análise do resultado

A BST manteve o índice de matrículas permanentemente ordenado ao longo de inserções e remoções, respondendo às consultas individuais sem percorrer todos os registros.

3.1.4 Situação Problema 8 - Sistema de autocomplete de nomes de produtos

Descrição e resultado da execução

A aplicação bst.AutocompleteProdutos armazena um catálogo de 30 produtos comerciais em uma BST indexada pelo nome normalizado e retorna sugestões ordenadas à medida que o usuário informa um prefixo.

Consulta no Autocomplete (30 Produtos)	Entrada	Resultado Observado
Autocomplete com prefixo válido	Prefixo 'ca'	6 sugestões: Caderno, Calculadora, Caneta Azul, Caneta Preta, Caneta Vermelha, Cartolina
Autocomplete com prefixo sem correspondência	Prefixo 'zzz'	Nenhuma sugestão retornada (subárvores podadas)
Busca exata de produto	'Caneta Azul'	Produto localizado corretamente no catálogo

Análise do resultado

A combinação das propriedades de ordem da BST com a poda de subárvores no método listarPorPrefixo permitiu recuperar apenas a faixa alfabética correspondente ao prefixo digitado, entregando as sugestões já em ordem alfabética.

3.1.5 Análise de complexidade

As operações inserir, buscar e remover possuem complexidade de tempo O(h), onde h é a altura da árvore: O(log n) no caso médio (árvore equilibrada) e O(n) no pior caso (inserção degenerada em ordem monótona). O percurso em-ordem visita todos os nós em O(n), enquanto listarPorPrefixo opera em O(h + k) para retornar k sugestões. O espaço ocupado é O(n) para os nós e O(h) na pilha de recursão.

3.1.6 Testes realizados e considerações sobre as aplicações

A suíte bst.TesteBST executou e aprovou 14 casos de teste, cobrindo árvore vazia, chaves duplicadas, remoção de folhas, remoção de nós com 1 e 2 filhos, remoção da raiz e consultas de autocomplete com diferentes prefixos.

3.2 ÁRVORE BALANCEADA (AVL)

3.2.1 Conceito e justificativa da escolha

A árvore AVL é uma árvore binária de busca autobalanceada na qual, para todo nó, a diferença de altura entre suas subárvores esquerda e direita (fator de balanceamento) pertence obrigatoriamente ao conjunto {-1, 0, 1}. Quando uma inserção ou remoção viola esse invariante, rotações locais simples (LL, RR) ou duplas (LR, RL) restauram o equilíbrio.

Em sistemas críticos como o índice de CPF de Recursos Humanos (onde cadastros podem chegar ordenados) e o ranking online de jogadores com atualizações constantes, a AVL evita a degeneração da BST comum para O(n) e garante desempenho estrito O(log n) até no pior caso.

3.2.2 Implementação da estrutura

Implementada na classe AVL<K extends Comparable<K>, V> (java(Cássio)/avl/AVL.java). Cada nó mantém sua altura atualizada ao final de cada chamada recursiva. O método rebalancear inspeciona o fator de balanceamento altura(esq) - altura(dir) e aplica rotacionarDireita (caso LL), rotacionarEsquerda (caso RR) ou rotações duplas esquerda-direita (LR) e direita-esquerda (RL). Além de inserir, buscar e remover, a classe oferece altura(), fatorBalanceamento(), estaBalanceada() e listarMaiores(int k).

3.2.3 Situação Problema 9 - Índice de CPF para sistema de Recursos Humanos

Descrição e resultado da execução

A aplicação avl.IndiceCpf insere 25 funcionários em ordem estritamente crescente de CPF (731.205.100-00 a 731.205.124-68). Em uma BST comum, essa entrada produziria uma lista degenerada de altura 25; na AVL, as rotações automáticas mantêm a árvore compacta.

Indicador da Simulação (Índice de CPF na AVL)	Resultado Observado
Funcionários cadastrados em ordem crescente de CPF	25 funcionários
Altura final da árvore AVL (vs. 25 na BST comum)	5 níveis
Fator de balanceamento da raiz	0
Verificação recursiva de todos os nós (estaBalanceada)	true (100% dos nós com fator entre -1 e +1)
Busca rápida por CPF (731.205.100-00)	Encontrado: Ana Lima — Cargo: Analista

Análise do resultado

A inserção sequencial comprovou na prática a eficácia das rotações da AVL: mesmo no pior cenário possível para uma árvore binária comum, a estrutura manteve altura 5 (log2(25) ≈ 4,64) e fator de balanceamento zero na raiz.

3.2.4 Situação Problema 10 - Ranking de jogadores em um jogo online

Descrição e resultado da execução

A aplicação avl.RankingJogadores mantém 30 jogadores ordenados na AVL por uma chave composta (%010d:%s) que combina pontuação formatada com 10 dígitos e nome, preservando jogadores empatados. Ao atualizar a pontuação de um jogador, a chave antiga é removida e a nova é inserida com rebalanceamento imediato. A consulta Top-K percorre a árvore em ordem reversa (direita -> raiz -> esquerda) visitando apenas os nós necessários.

Posição no Top-5	Jogador (após 3 atualizações da rodada)	Pontuação
1º Lugar	Marina Barros (atualizada de 664 para 920 pts)	920 pontos
2º Lugar	Isabela Dias	816 pontos
3º Lugar	Rafaela Torres	812 pontos
4º Lugar	Eduarda Lopes	808 pontos
5º Lugar	Heitor Melo	779 pontos
Estado da AVL	30 jogadores ativos | Altura: 6 | Fator da raiz: -1	Balanceada: true

Análise do resultado

A chave composta solucionou o problema de empates de pontuação (como Ana Lima e Fabio Santos empatados com 750 pontos após a rodada), enquanto o percurso reverso podado permitiu extrair os Top-5 imediatamente.

3.2.5 Análise de complexidade

Graças ao invariante de altura h = O(log n), as operações inserir, buscar e remover possuem complexidade de tempo O(log n) garantida no melhor, médio e pior casos. As rotações locais custam O(1). A consulta Top-K possui custo O(log n + K). O consumo de espaço é O(n) para os nós e O(log n) na pilha de recursão.

3.2.6 Testes realizados e considerações sobre as aplicações

A suíte avl.TesteAVL aprovou todos os 15 casos de teste, exercitando explicitamente as rotações simples e duplas (LL, RR, LR, RL), remoções sucessivas com rebalanceamento, empates no ranking e validação dos 25 CPFs e 30 jogadores.

3.3 GRAFOS (GRAPHS)

3.3.1 Conceito e justificativa da escolha

Grafos são estruturas formadas por um conjunto de vértices (V) e um conjunto de arestas (E) que conectam pares de vértices, podendo ser direcionados ou não-direcionados, ponderados ou não-ponderados. São a abstração matemática exata para representar malhas viárias e redes de relacionamento.

Na logística de entregas (Situação 11), um grafo ponderado representado por lista de adjacência combinado ao algoritmo de Dijkstra determina a rota de menor distância acumulada entre o Centro de Distribuição e qualquer bairro. Na rede social (Situação 12), um grafo não-direcionado combinado à Busca em Largura (BFS) determina conexões diretas, amigos em comum, recomendações de amizade e o menor grau de separação entre dois usuários.

3.3.2 Implementação da estrutura

Implementados em C no diretório c(Lukas)/Grafo/. Em c(Lukas)/Grafo/Entregas.c, a estrutura GrafoDelivery mantém um vetor de nomes de bairros e um vetor de ponteiros adj[] para listas encadeadas de nós NoAdj (contendo destino, peso e proximo). As rotas urbanas são cadastradas em mão dupla com pesos em quilômetros. A função dijkstra inicializa as distâncias com infinito (INF = 999999), seleciona iterativamente o vértice não visitado de menor distância acumulada, relaxa suas arestas adjacentes (dist[u] + peso < dist[v]) e reconstrói a cadeia completa de bairros através do vetor anterior[].

Em c(Lukas)/Grafo/rede_social.c, a estrutura RedeSocial utiliza uma matriz de adjacência simétrica (matriz[u][v] = matriz[v][u] = 1) para até 25 usuários (MAX = 25). Foram implementadas as operações conectarAmigos, listarAmigos, amigosEmComum (interseção das linhas da matriz em O(V)), sugerirConexoes (identificando amigos de amigos no 2º nível ainda não conectados) e grauSeparacaoBFS, que utiliza uma fila FIFO e os vetores distancia[] e anterior[] para encontrar o caminho mínimo em número de arestas.

3.3.3 Situação Problema 11 - Roteamento de entrega de mercadorias (Caminho Mínimo)

Descrição e resultado da execução

A simulação de c(Lukas)/Grafo/Entregas.c (Opção 4) modela um mapa urbano com 10 bairros + CD Central = 11 vértices interligados por 14 vias ponderadas em mão dupla e executa o algoritmo de Dijkstra entre cinco pares distintos de origem e destino.

Origem -> Destino (10 Bairros + CD Central = 11 Vértices)	Custo Mínimo	Menor Rota Calculada pelo Algoritmo de Dijkstra
CD Central -> Campus Universitário	22 km	CD Central -> Distrito Industrial -> Vila Nova -> Bela Vista -> Morumbi -> Parque das Flores -> Campus Universitário
CD Central -> Morumbi	13 km	CD Central -> Distrito Industrial -> Vila Nova -> Bela Vista -> Morumbi
Distrito Industrial -> Santa Cecília	14 km	Distrito Industrial -> Vila Nova -> São José -> Santa Cecília
Jardim Europa -> Parque das Flores	12 km	Jardim Europa -> Bela Vista -> Morumbi -> Parque das Flores
Centro -> Santa Cecília	16 km	Centro -> Vila Nova -> São José -> Santa Cecília

Análise do resultado

O algoritmo de Dijkstra identificou rotas não triviais de menor quilometragem: por exemplo, entre CD Central e Morumbi, em vez de seguir pela rota de maior peso passando pelo Centro e Jardim Europa (CD Central -> Centro -> Jardim Europa -> Bela Vista -> Morumbi = 16 km), selecionou o caminho ótimo de 13 km passando pelo Distrito Industrial, Vila Nova e Bela Vista.

3.3.4 Situação Problema 12 - Rede de amizades em uma rede social simples

Descrição e resultado da execução

A simulação de c(Lukas)/Grafo/rede_social.c (Opção 6) cadastra 15 usuários (Ana a Olivia) conectados por 16 relações de amizade e executa listagens de amigos diretos, interseção de amigos em comum, recomendação de novas conexões e cálculo de graus de separação via BFS.

Consulta na Rede Social (15 Usuários)	Parâmetros	Resultado Observado
Amigos diretos	Ana	Amigos diretos de Ana: Bruno, Carla e Daniel
Amigos em comum	Ana e Bruno / Elisa e Fabio	Entre Ana e Bruno: Carla | Entre Elisa e Fabio: Heitor
Sugestões de amizade (2º grau)	Para Ana	Elisa (1 em comum), Fabio (1 em comum) e Gabriela (1 em comum)
Grau de separação via BFS	Ana -> Heitor	3 graus (Ana -> Bruno -> Elisa -> Heitor)
Grau de separação via BFS	Ana -> Olivia	6 graus (Ana -> Daniel -> Gabriela -> Isabela -> Karen -> Marina -> Olivia)
Grau de separação via BFS	Bruno -> Nicolas	5 graus (Bruno -> Elisa -> Heitor -> Joao -> Lucas -> Nicolas)

Análise do resultado

A exploração em largura (BFS) por camadas de distância garantiu a descoberta da menor cadeia social entre quaisquer dois usuários, além de alimentar a lógica de recomendação de amigos de amigos.

3.3.5 Análise de complexidade

No roteamento de entregas com lista de adjacência, o armazenamento ocupa O(V + E) de espaço e o algoritmo de Dijkstra executa em O(V² + E) de tempo. Na rede social com matriz de adjacência, o espaço ocupado é O(V²), a verificação de amigos em comum custa O(V) e a busca em largura (BFS) opera em O(V²).

3.3.6 Testes realizados e considerações sobre as aplicações

Os módulos em C foram verificados a partir das simulações embutidas (Opção 4 em Entregas.c e Opção 6 em rede_social.c) e de consultas manuais nos menus interativos para casos de origem igual ao destino, vértices desconectados, nomes inexistentes, prevenção de auto-amizade, arestas duplicadas e liberação das listas de adjacência.

3.4 TABELA HASH (HASH TABLE)

3.4.1 Conceito e justificativa da escolha

A Tabela Hash é uma estrutura de indexação que mapeia uma chave a um índice de um vetor de compartimentos (buckets) por meio de uma função de espalhamento (hash function), proporcionando inserção, busca e remoção em tempo constante médio O(1).

Em um sistema web de autenticação de usuários (Situação 13), localizar a conta pelo e-mail e validar o hash da senha em O(1) médio é indispensável para resposta imediata. Na contagem de frequência de palavras em textos (Situação 14), a tabela hash permite consultar e incrementar o contador de cada vocábulo em tempo constante médio à medida que o texto é lido.

3.4.2 Implementação da estrutura

Implementadas em C no diretório c(Lukas)/Tabela_Hash/ com função de espalhamento polinomial de base 31 ((hash * 31 + char) % TAM_HASH) e tratamento de colisões por encadeamento externo (listas simplesmente encadeadas em cada bucket). Em c(Lukas)/Tabela_Hash/login.c (TAM_HASH = 11 para forçar e demonstrar colisões com 20 usuários), cada nó Usuario armazena email, usuario, senhaHash (calculado pelo algoritmo não criptográfico DJB2 com semente 5381 em gerarHashSenha, adotado com finalidade estritamente didática para ilustrar a comparação de resumos numéricos em vez de texto puro, visto que sistemas reais em produção exigem algoritmos criptográficos dedicados de derivação de chave com salt, como Argon2, bcrypt ou PBKDF2; o valor numérico do hash é mantido apenas internamente para autenticação, sem ser exposto no console ou na listagem de usuários) e o ponteiro proximo. As funções cadastrarUsuario, autenticar e removerUsuario calculam diretamente idx = funcaoHash(email) para atuar apenas sobre o bucket correspondente.

Em c(Lukas)/Tabela_Hash/frequencia.c (TAM_HASH = 31), a função processarTexto tokeniza o documento bruto, converte caracteres alfabéticos para minúsculas ignorando pontuações e aciona inserirPalavra, que incrementa o contador se a chave já existir no bucket ou aloca um novo nó Palavra.

3.4.3 Situação Problema 13 - Cadastro de usuários com login rápido

Descrição e resultado da execução

A simulação de c(Lukas)/Tabela_Hash/login.c (Opção 5) cadastra 20 usuários nos 11 buckets da tabela, exibe o mapa de distribuição das listas encadeadas e executa bateria de testes de autenticação e remoção de conta.

Teste Executado (Tabela Hash de Login)	Entrada	Resultado Observado
Cadastro de 20 usuários em 11 buckets	ana.lima@... a victor.braga@...	20 ativos; 5 buckets com colisões tratadas por encadeamento (buckets 3, 4, 7, 8 e 9)
1. Login válido no início do bucket	ana.lima@empresa.com / Senha#01	AUTENTICADO COM SUCESSO (acesso direto ao bucket)
2. Login válido em bucket com colisão	victor.braga@empresa.com / Senha#20	AUTENTICADO COM SUCESSO (percorrida lista do bucket)
3. Login com senha incorreta	ana.lima@empresa.com / errada123	FALHA (hash DJB2 divergente — esperado)
4. Login de e-mail inexistente	nao.existe@empresa.com / 123	FALHA (usuário não encontrado no bucket — esperado)
5. Remoção de conta e novo login	carla.alves@empresa.com	Conta removida do bucket 8 em O(1) médio; login subsequente recusado

Análise do resultado

Com fator de carga α = 20/11 ≈ 1,82, ocorreram colisões em 5 buckets que foram absorvidas sem perda de dados pelo encadeamento externo. A autenticação e a remoção acessaram exclusivamente o bucket indicado pela função hash, preservando o custo médio O(1).

3.4.4 Situação Problema 14 - Contagem de frequência de palavras em um texto

Descrição e resultado da execução

A simulação de c(Lukas)/Tabela_Hash/frequencia.c (Opção 2) processa um texto técnico de demonstração sobre estruturas de dados e tabelas hash armazenado no próprio código, contabilizando todas as ocorrências na tabela de dispersão e exibindo as palavras mais frequentes e o total de vocábulos distintos.

Indicador / Ranking do Texto Processado	Resultado Observado
Total de palavras processadas no texto de demonstração	103 palavras
Total de palavras distintas (chaves únicas na Hash)	67 palavras distintas
Top 1º a 3º termos mais frequentes	1º 'de' (12x), 2º 'e' (6x), 3º 'dados' (4x), 4º 'hash' (4x)
Top 4º a 8º termos mais frequentes	5º 'a' (3x), 6º 'em' (3x), 7º 'na' (3x), 8º 'tabela' (3x), 9º 'uma' (3x)
Top 9º a 12º termos mais frequentes	10º 'boa' (2x), 11º 'chaves' (2x), 12º 'estrutura' (2x)

Análise do resultado

A tabela hash consolidou as 103 palavras do texto técnico de demonstração em 67 entradas distintas em uma única passagem pelo texto, demonstrando sua eficiência na construção de índices de frequência lexical.

3.4.5 Análise de complexidade

Com distribuição uniforme e fator de carga controlado (α = N/M), as operações de inserção, busca/autenticação e remoção possuem complexidade de tempo O(1 + α) = O(1) no caso médio, e O(N) no pior caso degenerado em que todas as chaves colidem no mesmo bucket. O processamento completo de um texto de P palavras tem custo médio O(P). A complexidade de espaço é O(M + N), onde M é o número de buckets e N o número de elementos distintos armazenados.

3.4.6 Testes realizados e considerações sobre as aplicações

A verificação dos programas em C combinou a execução das baterias demonstrativas do menu (Opção 5 em login.c e Opção 2 em frequencia.c) com testes interativos de bloqueio de cadastro duplicado para e-mail já existente ([ERRO] E-mail ja cadastrado), colisões múltiplas no mesmo bucket, remoção de nós no início e no meio da lista encadeada do bucket, verificação da não exposição do hash numérico das senhas nas saídas do console, normalização de maiúsculas/minúsculas e liberação de memória.

4. ALGORITMOS DE ORDENAÇÃO

Os algoritmos de ordenação reorganizam coleções de registros segundo critérios de comparação bem definidos (crescente, decrescente ou múltiplos campos), constituindo a base para visualização classificada, geração de rankings e buscas eficientes.

4.1 BUBBLE SORT

4.1.1 Conceito e justificativa da escolha

O Bubble Sort percorre repetidamente o vetor comparando elementos adjacentes e permutando-os quando estão fora de ordem, fazendo com que os maiores valores 'borbulhem' para o final a cada passagem. É um algoritmo estável, in-place e de implementação simples, indicado para conjuntos pequenos (como notas de uma turma de até 30 alunos ou vitrines enxutas) e para análise comparativa de operações elementares.

4.1.2 Implementação do algoritmo

Implementado na classe BubbleSort (java(Cássio)/bubblesort/BubbleSort.java) com suporte a Comparator<T>, flag booleana trocou para parada antecipada caso uma passagem termine sem permutações e método ordenarComEstatisticas, que contabiliza o total exato de comparações e trocas realizadas.

4.1.3 Situação Problema 15 - Ordenação de notas de uma turma pequena

Descrição e resultado da execução

A aplicação bubblesort.NotasTurma ordena os registros (nome + nota) de 20 alunos, calcula a média da turma e compara diretamente o esforço computacional do Bubble Sort otimizado com o do Insertion Sort sobre a mesma entrada.

Indicador / Métrica de Desempenho (20 Alunos)	Resultado Observado
Quantidade de alunos / Média da turma	20 alunos | Média: 5,05 (menor: Ana Lima 0,0 | maior: Isabela Dias 10,0)
Bubble Sort Otimizado (parada antecipada)	184 comparações | 87 trocas
Insertion Sort (mesma entrada de 20 notas)	106 comparações | 87 deslocamentos
Economia do Insertion Sort frente ao Bubble Sort	78 comparações a menos (redução de 42,4% nas comparações)

Análise do resultado

Embora ambos os algoritmos pertençam à classe quadrática O(n²) e realizem 87 movimentações (número exato de inversões da sequência), o Insertion Sort interrompe a varredura interna assim que encontra a posição correta de cada nota, economizando 78 comparações em relação ao Bubble Sort.

4.1.4 Situação Problema 16 - Ordenação de produtos por preço em uma vitrine pequena

Descrição e resultado da execução

A aplicação bubblesort.ProdutosPreco organiza uma vitrine de 12 produtos permitindo alternar entre ordem crescente e ordem decrescente de preço.

Modalidade de Ordenação (12 Produtos)	Primeiro Item da Vitrine	Último Item da Vitrine
Ordem Crescente de Preço	Cabo HDMI — R$ 32,50	Monitor — R$ 899,90
Ordem Decrescente de Preço	Monitor — R$ 899,90	Cabo HDMI — R$ 32,50

Análise do resultado

Para 12 itens, o tempo de resposta do Bubble Sort é instantâneo e a parametrização via comparador permite inverter a vitrine sem duplicar a lógica do algoritmo.

4.1.5 Análise de complexidade

Com a flag de parada antecipada, o Bubble Sort atinge complexidade de tempo O(n) no melhor caso (vetor já ordenado, exigindo n - 1 comparações e 0 trocas). Nos casos médio e pior, realiza O(n²) comparações e O(n²) trocas. A complexidade de espaço auxiliar é O(1).

4.1.6 Testes realizados e considerações sobre as aplicações

A suíte bubblesort.TesteBubbleSort aprovou 13 casos de teste, incluindo validação da estabilidade, parada antecipada em O(n), vetores inversos e conferência da comparação estatística com o Insertion Sort.

4.2 SELECTION SORT

4.2.1 Conceito e justificativa da escolha

O Selection Sort divide o vetor em uma sublista já ordenada e uma sublista restante: a cada passo i, localiza o índice do menor (ou maior) elemento entre [i..n-1] e realiza uma única troca direta com a posição i. Sua grande vantagem sobre o Bubble Sort é executar no máximo n - 1 trocas — complexidade linear O(n) de escritas na memória —, sendo ideal quando se deseja selecionar os Top-N candidatos em um processo seletivo ou reorganizar listas diárias de tarefas minimizando movimentações de registros.

4.2.2 Implementação do algoritmo

Implementado em C no diretório c(Lukas)/Selection_Sort/. Em processo_seletivo.c, a função selectionSortTopN ordena o vetor de registros Candidato (nome e nota) em ordem decrescente, contabilizando comparações e trocas e destacando os Top-N aprovados. Em tarefas.c, a função selectionSortTarefas ordena registros Tarefa pela prioridade numérica (1 = Crítica/Urgente até 5 = Baixa).

4.2.3 Situação Problema 17 - Seleção dos melhores candidatos em um processo seletivo

Descrição e resultado da execução

A simulação de c(Lukas)/Selection_Sort/processo_seletivo.c (Opção 3) avalia 15 candidatos que realizaram uma prova técnica e seleciona os Top-5 melhores colocados.

Classificação Top-5 (15 Candidatos)	Candidato Selecionado	Pontuação Obtida
1º Lugar (Aprovado)	Elisa Sousa	95,00 pontos
2º Lugar (Aprovado)	Isabela Dias	93,50 pontos
3º Lugar (Aprovado)	Bruno Costa	91,00 pontos
4º Lugar (Aprovado)	Lucas Martins	90,00 pontos
5º Lugar (Aprovado)	Nicolas Freitas	89,00 pontos
Estatísticas do Selection Sort	105 comparações (n(n-1)/2 para n=15)	Apenas 13 trocas realizadas

Análise do resultado

Para classificar os 15 candidatos, o algoritmo realizou exatamente 15 × 14 / 2 = 105 comparações e apenas 13 trocas efetivas de estruturas, posicionando imediatamente os 5 candidatos de maior nota no topo da lista.

4.2.4 Situação Problema 18 - Organização de tarefas por prioridade (Valor Numérico)

Descrição e resultado da execução

A simulação de c(Lukas)/Selection_Sort/tarefas.c (Opção 3) organiza um cronograma diário de 14 tarefas distribuídas entre os níveis de prioridade 1 (Urgente) a 5 (Baixa).

Faixa de Prioridade (14 Tarefas)	Exemplos de Tarefas Reorganizadas	Posições na Lista Final
Prioridade 1 (Urgente — 3 tarefas)	Corrigir falha crítica em produção; Aplicar patch de segurança; Restaurar backup	Posições 1ª a 3ª
Prioridade 2 (Alta — 4 tarefas)	Daily Scrum; Revisar Pull Request; Validar homologação; Testar API de pagamentos	Posições 4ª a 7ª
Prioridade 3 (Média — 3 tarefas)	Planejar backlog da Sprint; Responder e-mails de fornecedores; Elaborar relatório semanal	Posições 8ª a 10ª
Prioridade 4 (Normal — 2 tarefas)	Atualizar documentação interna do módulo; Pesquisar ferramentas de monitoramento	Posições 11ª a 12ª
Prioridade 5 (Baixa — 2 tarefas)	Limpar logs temporários de homologação; Organizar arquivos antigos da pasta	Posições 13ª a 14ª
Estatísticas da Ordenação	91 comparações (14 × 13 / 2)	Apenas 10 trocas realizadas

Análise do resultado

O Selection Sort agrupou todas as tarefas críticas no início do expediente com apenas 10 permutações de memória.

4.2.5 Análise de complexidade

O Selection Sort realiza sempre n(n - 1)/2 comparações, apresentando complexidade de tempo $O(n^2)$ no melhor, médio e pior casos. Em contrapartida, o número de trocas é O(n) (no máximo n - 1 trocas). A complexidade de espaço auxiliar é O(1).

4.2.6 Testes realizados e considerações sobre as aplicações

Os dois programas em C foram validados executando as simulações padrão (Opção 3 em processo_seletivo.c e Opção 3 em tarefas.c) e testes manuais via menu interativo com diferentes valores de Top-N (incluindo N igual ao total de candidatos), prioridades repetidas de 1 a 5 e listas unitárias.

4.3 INSERTION SORT

4.3.1 Conceito e justificativa da escolha

O Insertion Sort constrói a sequência ordenada incrementalmente, tomando cada elemento a partir da segunda posição e inserindo-o na posição correta dentro do prefixo já ordenado por meio do deslocamento dos elementos maiores para a direita. É estável, in-place e atinge tempo linear O(n) quando os dados já estão ordenados ou quase ordenados.

No jogo de cartas (Situação 19), representa exatamente o processo humano de inserir cada nova carta comprada na mão já ordenada. No ranking de vendedores (Situação 20), aproveita a ordenação prévia para atualizar a tabela diária com mínimo esforço.

4.3.2 Implementação do algoritmo

Implementado na classe InsertionSort (java(Cássio)/insertionsort/InsertionSort.java) com método ordenar e método ordenarComEstatisticas, que contabiliza cada comparação efetiva e cada deslocamento realizados no laço interno.

4.3.3 Situação Problema 19 - Inserção de novas cartas em um jogo de cartas ordenado

Descrição e resultado da execução

A aplicação insertionsort.JogoCartas simula o recebimento de 13 cartas fora de ordem (K de Copas, 3 de Paus, A de Espadas, 10 de Ouros, 7 de Copas, ..., 6 de Copas), exibindo o estado da mão ordenada por valor (Ás = 1 a Rei = 13, com desempate por naipe) após cada inserção.

Etapa da Simulação (Jogo de Cartas)	Carta Recebida	Estado da Mão Ordenada
1ª Carta recebida	K de Copas	K de Copas
3ª Carta recebida	7 de Copas	A de Espadas, 3 de Paus, K de Copas
5ª Carta recebida	A de Espadas	A de Espadas, 3 de Paus, 7 de Copas, 10 de Ouros, K de Copas
13ª Carta recebida (Mão Final)	6 de Copas	A de Espadas, 2 de Espadas, 3 de Paus, 4 de Espadas, 5 de Copas, 6 de Copas, 7 de Copas, 8 de Ouros, 9 de Paus, 10 de Ouros, J de Paus, Q de Ouros, K de Copas

Análise do resultado

A visualização passo a passo comprova a natureza incremental do Insertion Sort: como as cartas anteriores já estão ordenadas, cada nova carta desloca apenas as cartas maiores que ela até assumir sua posição exata.

4.3.4 Situação Problema 20 - Atualização de ranking de vendedores do mês

Descrição e resultado da execução

A aplicação insertionsort.RankingVendedores acompanha 10 vendedores ao longo de 7 dias de vendas acumuladas e, ao final, compara o número de comparações do Insertion Sort entre o ranking já ordenado e o mesmo ranking embaralhado (semente fixa 42).

Cenário / Indicador (10 Vendedores — 7 Dias)	Resultado Observado
Líder ao final do Dia 7 (1º lugar)	Isabela Dias — R$ 4.259,50 acumulados
Vice-líder ao final do Dia 7 (2º lugar)	Bruno Costa — R$ 4.127,50 acumulados
Desempenho em lista já ordenada (melhor caso O(n))	9 comparações (n - 1 comparações para n = 10) e 0 deslocamentos
Desempenho em lista embaralhada (semente 42 — caso médio $O(n^2)$)	31 comparações observadas (3,44x mais comparações)

Análise do resultado

A comparação experimental confirma a adaptabilidade do Insertion Sort: quando a lista já se encontra ordenada, o laço interno falha imediatamente na primeira verificação de cada passo, executando exatamente n - 1 = 9 comparações, contra 31 comparações na entrada embaralhada.

4.3.5 Análise de complexidade

A complexidade de tempo é O(n) no melhor caso (entrada ordenada) e $O(n^2)$ no caso médio e no pior caso (entrada em ordem inversa). A complexidade de espaço auxiliar é O(1).

4.3.6 Testes realizados e considerações sobre as aplicações

A suíte insertionsort.TesteInsertionSort aprovou 12 casos de teste, verificando estabilidade, contagem exata de comparações no melhor e pior casos, ordenação das 13 cartas e simulação semanal dos vendedores.

4.4 MERGE SORT

4.4.1 Conceito e justificativa da escolha

O Merge Sort aplica o paradigma de divisão e conquista: divide recursivamente o vetor ao meio até subproblemas unitários e intercala (merge) as metades ordenadas em tempo linear. Diferencia-se por oferecer garantia estrita de tempo O(n log n) mesmo no pior caso e por ser naturalmente estável.

No histórico de transações bancárias (Situação 21), o volume de lançamentos exige O(n log n) previsível e estabilidade para manter a ordem original de registro de transações ocorridas no mesmo minuto. Na integração de contatos de dois departamentos (Situação 22), a etapa de intercalação combina duas listas já ordenadas em tempo linear O(n + m) eliminando duplicatas.

4.4.2 Implementação do algoritmo

Implementado na classe MergeSort (java(Cássio)/mergesort/MergeSort.java). O método intercalar preserva a estabilidade ao selecionar o elemento da sublista esquerda sempre que comparador.compare(esquerda, direita) <= 0. O método mesclarContatos aplica o avanço simultâneo de dois ponteiros sobre as listas ordenadas dos departamentos A e B, descartando nomes duplicados (case-insensitive) em uma única passagem.

4.4.3 Situação Problema 21 - Ordenação de histórico de transações bancárias

Descrição e resultado da execução

A aplicação mergesort.TransacoesBancarias ordena cronologicamente um extrato com 50 transações bancárias (PIX recebidos e pagamentos de compras) contendo registros com timestamps idênticos.

Ordem Cronológica	Data e Hora	Valor e Descrição (Estabilidade Preservada)
1º Registro	01/08/2026 08:00	R$ 25,00 — PIX recebido #1 (mantido antes do #26)
2º Registro	01/08/2026 08:00	R$ -500,00 — Pagamento de compra #26
3º Registro	01/08/2026 09:00	R$ -154,00 — Pagamento de compra #4 (mantido antes do #29)
4º Registro	01/08/2026 09:00	R$ 29,00 — PIX recebido #29
50º Registro	02/08/2026 08:00	R$ -246,00 — Pagamento de compra #48 (após #23 do mesmo horário)

Análise do resultado

O Merge Sort organizou as 50 transações por data/hora e preservou intacta a ordem relativa de lançamento de todos os pares de transações registrados no mesmo minuto (#1 antes de #26, #4 antes de #29, #23 antes de #48).

4.4.4 Situação Problema 22 - Mesclagem de duas listas de contatos ordenadas

Descrição e resultado da execução

A aplicação mergesort.MesclagemContatos recebe duas listas alfabeticamente ordenadas de 20 contatos cada (Departamento A e Departamento B, com 10 colaboradores em comum) e gera uma lista unificada com duplicatas

Indicador da Mesclagem (Contatos)	Resultado Observado
Contatos de entrada (Departamento A + Departamento B)	20 + 20 = 40 registros ordenados
Duplicatas detectadas e eliminadas durante o merge	10 nomes em comum (mantida a ocorrência do Depto. A)
Total final de contatos únicos ordenados	30 contatos únicos (de Alice Teixeira a Victor Braga)

Análise do resultado

Ao explorar a pré-ordenação das duas entradas diretamente na rotina de intercalação, a mesclagem deduplicada foi concluída em tempo linear O(n + m), sem executar uma nova ordenação completa.

4.4.5 Análise de complexidade

O Merge Sort possui complexidade de tempo O(n log n) no melhor, médio e pior casos, e complexidade de espaço auxiliar O(n) para o vetor temporário (além de O(log n) na pilha de recursão). A rotina de mesclagem direta de duas listas ordenadas de tamanhos n e m possui complexidade de tempo O(n + m) e espaço O(n + m) para o vetor resultante.

4.4.6 Testes realizados e considerações sobre as aplicações

A suíte mergesort.TesteMergeSort aprovou todos os 16 casos de teste, cobrindo estabilidade estrita em objetos equivalentes, vetores de diferentes tamanhos, mesclagem com listas vazias/idênticas/disjuntas e validação das 50 transações e dos 30 contatos.

4.5 QUICK SORT

4.5.1 Conceito e justificativa da escolha

O Quick Sort é um algoritmo de ordenação por divisão e conquista que seleciona um elemento pivô, particiona o vetor in-place posicionando os elementos menores ou iguais à esquerda do pivô e os maiores à direita, e ordena recursivamente as duas partições. Na prática, é o algoritmo mais veloz em memória principal graças à excelente localidade de cache, constantes multiplicativas reduzidas e uso de memória auxiliar restrito à pilha de recursão O(log n).

Neste trabalho, as duas aplicações em C utilizam estratégias distintas de escolha de pivô no Quick Sort: no catálogo de e-commerce de 100 produtos com múltiplos critérios (Situação 23 — c(Lukas)/Quick_Sort/ecommerce.c), implementou-se a escolha de pivô por mediana-de-três (mediana entre o primeiro, o central e o último elemento da partição) para reduzir a probabilidade de que entradas já ordenadas ou inversamente ordenadas provoquem a degradação para o pior caso O(n²); já na classificação de 30 atletas em competição esportiva (Situação 24 — c(Lukas)/Quick_Sort/competicao.c), utilizou-se o particionamento clássico de Lomuto com o último elemento da partição como pivô.

4.5.2 Implementação do algoritmo

Implementado em C no diretório c(Lukas)/Quick_Sort/. Em ecommerce.c, a função escolherPivoMedianaDeTres ordena entre si v[low], v[mid] e v[high] segundo o critério ativo (1 = Preço crescente, 2 = Nome A-Z, 3 = Popularidade decrescente) e posiciona a mediana como pivô da função particionar. O módulo também inclui uma implementação completa do Merge Sort (mergeSortRec) para realizar o benchmark comparativo de tempo e número de operações sobre os mesmos 100 produtos. Em c(Lukas)/Quick_Sort/competicao.c, o Quick Sort (com particionamento de Lomuto tomando o último elemento v[high] como pivô) ordena registros Atleta pelo tempo de prova em segundos (menor tempo vence) e destaca o Pódio Oficial (Ouro, Prata e Bronze).

4.5.3 Situação Problema 23 - Ordenação de catálogo de produtos de e-commerce

Descrição e resultado da execução

A simulação de c(Lukas)/Quick_Sort/ecommerce.c (Opção 3) gera 100 produtos comerciais de tecnologia, ordena o catálogo pelos três critérios solicitados (Preço, Nome e Popularidade) e executa o comparativo direto entre Quick Sort (Mediana-de-Três) e Merge Sort.

Critério / Benchmark (Catálogo de 100 Produtos)	Resultado e Estatísticas Observadas
1. Ordenação por Preço Crescente (Quick Sort)	1º: [ID 53] Placa de Video Modelo 025 (R$ 76,70) ... 100º: [ID 26] Roteador Modelo 026 (R$ 4.408,40)
2. Ordenação por Nome Alfabético A-Z (Quick Sort)	1º: [ID 74] Headset Modelo 002 (R$ 3.710,60) ... 100º: [ID 9] Webcam Modelo 097 (R$ 1.470,10)
3. Ordenação por Popularidade Decrescente (Quick Sort)	1º: [ID 51] Mouse Modelo 051 (Pop: 993) ... 100º: [ID 8] Teclado Modelo 060 (Pop: 100)
Benchmark Quick Sort (Mediana-de-3) — 100 itens	999 comparações (incluindo mediana-de-3) | 493 trocas | Pilha O(log n) médio
Benchmark Merge Sort — mesmos 100 itens	500 comparações | Requer vetor auxiliar O(n) para intercalação

Análise do resultado

O benchmark evidencia o trade-off clássico entre os dois algoritmos O(n log n): o Merge Sort realiza menos comparações de chaves (500 vs. 999, já que a mediana-de-três realiza 3 comparações adicionais por partição para escolher um pivô equilibrado), porém exige alocação de vetor auxiliar O(n), enquanto o Quick Sort opera in-place com 493 trocas diretas.

4.5.4 Situação Problema 24 - Classificação de resultados de uma competição esportiva

Descrição e resultado da execução

A simulação de c(Lukas)/Quick_Sort/competicao.c (Opção 3) classifica o resultado oficial de uma corrida de 100 metros rasos com 30 atletas, identificando o Pódio (Top 3) e a tabela completa.

Posição no Pódio / Classificação (30 Atletas)	Atleta	Tempo Oficial de Prova
OURO — 1º Lugar no Pódio	Victor Braga	9,91 segundos
PRATA — 2º Lugar no Pódio	Heitor Melo	9,98 segundos
BRONZE — 3º Lugar no Pódio	Lucas Martins	10,05 segundos
4º ao 30º Colocados	4º Henrique Vieira (10,08 s), 5º Daniel Rocha (10,12 s) ... 30º Karen Ribeiro (12,45 s)	Ordenados em 121 comparações e 88 particionamentos/trocas

Análise do resultado

O Quick Sort classificou os 30 atletas rapidamente com apenas 121 comparações e 88 trocas, permitindo extrair imediatamente os medalhistas nas posições 0, 1 e 2 do vetor ordenado.

4.5.5 Análise de complexidade

O Quick Sort apresenta complexidade de tempo O(n log n) no melhor caso e no caso médio. No pior caso (particionamentos severamente desbalanceados), a complexidade permanece O(n²); a escolha do pivô por mediana-de-três reduz significativamente a probabilidade dessa degradação em padrões comuns de entrada (como sequências já ordenadas ou inversamente ordenadas), mas não elimina o pior caso teórico O(n²). A complexidade de espaço auxiliar é O(log n) no caso médio para a pilha de chamadas recursivas (podendo atingir O(n) no pior caso sem eliminação de cauda).

4.5.6 Testes realizados e considerações sobre as aplicações

A validação foi conduzida por meio da rotina de simulação automatizada do módulo (que constrói os 100 produtos e os 30 atletas, executa as ordenações por preço, nome, popularidade e tempo de prova, e contabiliza comparações e trocas frente ao Merge Sort) complementada por testes manuais via menu interativo com entradas já ordenadas, valores empatados e coleções pequenas.

5. SÍNTESE DE COMPLEXIDADE, DIFICULDADES E CONCLUSÃO GERAL

5.1 Síntese Comparativa de Complexidade e Validação

A tabela a seguir consolida a análise assintótica de tempo (Melhor, Médio e Pior casos) e de espaço auxiliar de todas as 12 estruturas de dados e algoritmos de ordenação implementados neste trabalho:

Cat.	Estrutura / Algoritmo	Tempo (Melhor / Médio / Pior Caso)	Espaço Auxiliar	Aplicações Desenvolvidas
2.1	Fila Circular (FIFO)	O(1) / O(1) / O(1) nas extremidades	O(1) aux / O(N) fixo	Clínica Médica (15) e Impressão (12)
2.2	Pilha (LIFO)	O(1) push-pop / O(n) expressão	O(n)	Undo/Redo (12) e Expressões Matemáticas
2.3	Lista Encadeada	O(1) início / O(n) busca e meio	O(n) dinâmico	Playlist (12 músicas) e Estoque (7 dias)
3.1	Árvore Binária (BST)	O(log n) / O(log n) / O(n)	O(h) recursão	Cadastro Alunos (20) e Autocomplete (30)
3.2	Árvore Balanceada (AVL)	O(log n) / O(log n) / O(log n)	O(log n) recursão	Índice CPF (25) e Ranking Jogadores (30)
3.3	Grafos (Dijkstra / BFS)	Dijkstra O(V²+E) | BFS O(V²)	O(V+E) / O(V²)	Entregas (10 bairros + CD = 11 vértices) e Rede Social (15)
3.4	Tabela Hash	O(1) / O(1) / O(n) por chave	O(M + N)	Login Rápido (20) e Frequência (Texto de Demonstração)
4.1	Bubble Sort Otimizado	O(n) / O(n²) / O(n²)	O(1) (in-place)	Notas da Turma (20) e Vitrine (12)
4.2	Selection Sort	O(n²) / O(n²) / O(n²) [O(n) trocas]	O(1) (in-place)	Processo Seletivo (15) e Tarefas (14)
4.3	Insertion Sort	O(n) / O(n²) / O(n²)	O(1) (in-place)	Jogo de Cartas (13) e Vendedores (10)
4.4	Merge Sort	O(n log n) em todos os casos	O(n)	Transações (50) e Mesclagem Contatos (30)
4.5	Quick Sort (Mediana-3 / Lomuto)	O(n log n) / O(n log n) / O(n²)	O(log n) médio / O(n) pior caso (pilha)	E-commerce (100) e Competição (30 atletas)

Na parte em Java, todas as 7 suítes de testes unitários automatizados foram executadas pelo script verificar.sh, atingindo 100% de aprovação (107 casos de teste aprovados, 0 pendentes e 0 falhas). Na parte em C, os 12 programas foram compilados com GCC e tiveram seus resultados conferidos a partir das saídas reais das 12 rotinas de simulação demonstrativa integradas aos módulos, complementadas por testes manuais nos menus interativos:

Suíte de Teste Automatizado (Java) / Módulo (C)	Quantidade de Casos / Simulações	Status Final
apoio.TesteApoio (Validações, Entradas e Massa de Dados)	21 casos de teste	21 OK (0 falhas)
fila.TesteFila (Fila Circular, Clínica e Impressão)	16 casos de teste	16 OK (0 falhas)
bubblesort.TesteBubbleSort (NotasTurma e ProdutosPreco)	13 casos de teste	13 OK (0 falhas)
insertionsort.TesteInsertionSort (JogoCartas e RankingVendedores)	12 casos de teste	12 OK (0 falhas)
bst.TesteBST (CadastroAlunos e AutocompleteProdutos)	14 casos de teste	14 OK (0 falhas)
mergesort.TesteMergeSort (TransacoesBancarias e MesclagemContatos)	16 casos de teste	16 OK (0 falhas)
avl.TesteAVL (IndiceCpf e RankingJogadores)	15 casos de teste	15 OK (0 falhas)
Módulos em C (Pilha, Lista, Grafo, Hash, Selection e Quick Sort)	12 rotinas de simulação + testes via menu (Situações 3,4,5,6,11,12,13,14,17,18,23,24)	Saídas conferidas (GCC)
TOTAL CONSOLIDADO DO PROJETO	107 testes automatizados (Java) + 12 simulações/menus (C)	CONCLUÍDO

5.2 Dificuldades Encontradas e Soluções Adotadas

Durante o desenvolvimento e a integração das 24 aplicações do zero, a dupla enfrentou desafios técnicos importantes que foram solucionados da seguinte maneira:

1. Aritmética circular no cancelamento do último trabalho da Fila: Na implementação do cancelamento do último trabalho enviado na fila circular (removerUltimo), o recuo simples fim - 1 gerava índice negativo quando fim valia 0. O problema foi resolvido aplicando a fórmula modular (fim - 1 + elementos.length) % elementos.length, mantendo a operação em O(1).

2. Rastreamento da posição exata de erro na Pilha de expressões: Para atender ao requisito valorizado do relatório de erros na Situação 4, a pilha foi projetada para empilhar uma estrutura contendo simultaneamente o caractere de abertura e seu índice na string, permitindo diagnosticar a coluna exata tanto de fechamentos incompatíveis quanto de delimitadores abertos que nunca foram fechados.

3. Reordenação de nós e remoção automática na Lista Encadeada: Na playlist de músicas, a operação moverMusica exigiu desconectar o nó da posição de origem e religá-lo na posição de destino tratando corretamente o caso especial de mudança da cabeça da lista (inicio). No estoque, garantiu-se que a baixa de vendas para quantidade zero acionasse removerProduto com liberação segura via free().

4. Tratamento de empates de pontuação na Árvore AVL: Como árvores binárias de busca não admitem chaves duplicadas, jogadores com a mesma pontuação poderiam sobrescrever uns aos outros no ranking online. A solução adotada foi construir uma chave composta formatando a pontuação com zeros à esquerda seguida do nome (%010d:%s), preservando a ordem numérica e desempatando alfabeticamente sem perda de participantes.

5. Reconstrução de caminhos mínimos em Grafos (Dijkstra e BFS): Nos módulos de entregas e rede social, além de calcular a distância numérica mínima, era necessário exibir toda a cadeia de bairros ou amigos. Para isso, foram mantidos vetores de predecessores (anterior[]) durante o relaxamento das arestas, reconstruindo o trajeto completo da origem ao destino.

6. Garantia de busca O(1) médio e uso didático do hash DJB2 na Tabela Hash: No sistema de login, evitou-se qualquer varredura linear pelos buckets: tanto autenticar quanto removerUsuario aplicam funcaoHash(email) para inspecionar exclusivamente a lista encadeada do bucket correspondente. Para demonstrar o conceito de verificação por resumo numérico sem armazenar senhas em texto puro no exercício acadêmico, utilizou-se a função de espalhamento DJB2; ressalta-se, contudo, que o DJB2 é uma função hash rápida e não criptográfica, inadequada para segurança real de senhas em produção (cenário que requer funções criptográficas lentas com salt, como bcrypt, PBKDF2 ou Argon2).

7. Estabilidade e mitigação de casos desfavoráveis nas Ordenações: No Merge Sort e no Insertion Sort, o uso estrito da condição <= 0 garantiu estabilidade para transações bancárias registradas no mesmo minuto. No Quick Sort aplicado ao catálogo de 100 produtos, a seleção de pivô por mediana-de-três reduziu substancialmente a probabilidade de partições degeneradas em padrões comuns de entrada (como vetores previamente ordenados), embora não elimine o pior caso teórico $O(n^2)$ do algoritmo.

5.3 Conclusão sobre a Aplicabilidade das Estruturas em Problemas Reais

A implementação prática das 12 categorias e 24 situações-problema permitiu comprovar que a escolha adequada de uma estrutura de dados ou algoritmo de ordenação impacta diretamente a clareza, a escalabilidade e o desempenho de sistemas reais do cotidiano.

Constatou-se que as estruturas lineares são insubstituíveis quando o problema é governado pela ordem temporal ou posicional dos eventos: Filas asseguram justiça em atendimentos e spoolers de impressão (FIFO), Pilhas viabilizam reversão de histórico e validação sintática (LIFO) e Listas Encadeadas oferecem elasticidade para coleções mutáveis como playlists e estoques.

Por sua vez, quando o requisito central passa a ser a eficiência de busca ou a modelagem de conexões, as estruturas não-lineares tornam-se indispensáveis: BST e AVL mantêm índices ordenados permitindo consultas por prefixo e extração de Top-K em tempo logarítmico O(log n); Grafos com Dijkstra e BFS solucionam problemas logísticos de menor rota e graus de separação em redes sociais; e Tabelas Hash entregam autenticação e contagem lexical em tempo constante médio O(1).

Finalmente, o estudo comparativo dos cinco algoritmos de ordenação evidenciou que algoritmos quadráticos simples — como Insertion Sort em listas quase ordenadas (9 comparações para 10 vendedores) e Selection Sort quando se deseja minimizar escritas em memória — têm nichos reais bem definidos em conjuntos pequenos, enquanto algoritmos O(n log n) de divisão e conquista são fundamentais em volumes maiores: o Merge Sort quando se exige estabilidade estrita ou intercalação linear de bases ordenadas, e o Quick Sort (com mediana-de-três no e-commerce e particionamento de Lomuto na competição esportiva) quando se busca máxima velocidade in-place em memória principal.

6. REFERÊNCIAS BIBLIOGRÁFICAS

CELES, Waldemar; CERQUEIRA, Renato; RANGEL, José Lucas. Introdução a Estruturas de Dados: com Técnicas de Programação em C. 2. ed. Rio de Janeiro: Elsevier, 2016.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Algoritmos: Teoria e Prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

GOODRICH, Michael T.; TAMASSIA, Roberto; GOLDWASSER, Michael H. Estruturas de Dados e Algoritmos em Java. 6. ed. Porto Alegre: Bookman, 2015.

ISO/IEC. ISO/IEC 9899:2011 — Information technology — Programming languages — C (C11 Standard). Geneva: International Organization for Standardization, 2011.

ORACLE. Java Platform, Standard Edition 17 API Specification. Redwood Shores: Oracle Corporation, 2021. Disponível em: https://docs.oracle.com/en/java/javase/17/docs/api/. Acesso em: nov. 2026.

SEDGEWICK, Robert; WAYNE, Kevin. Algorithms. 4. ed. Boston: Addison-Wesley Professional, 2011.

SILVA, Hilson. Material Didático e Edital do Trabalho Prático Avaliativo (P2) — Disciplina de Algoritmos e Estruturas de Dados I. Teresina: Instituto de Ensino Superior iCEV, Curso de Engenharia de Software (Turma Snyder — 2026.2), 2026.

ZIVIANI, Nivio. Projeto de Algoritmos: com Implementações em Java e C++. 3. ed. São Paulo: Cengage Learning, 2011.